

Report: Final Line Detection Pipeline

White tape detection on a dark floor using MATLAB image processing

1. Overview

The purpose of the code is to detect a white masking-tape line on a dark floor from a recorded video. The code compares different segmentation approaches, then uses the most reliable and practical method as the final pipeline. The final version uses grayscale thresholding because the tape is mainly distinguished from the floor by brightness rather than by colour. After detecting the tape, the code calculates the tape centroid and converts its horizontal position into a normalised line-position error signal.

The output figures show that grayscale and RGB-brightness segmentation detect the line almost identically. The HSV mask, using the selected thresholds $S < 0.25$ and $V > 0.70$, fails to detect the tape in the tested frames because the tape is not bright enough under the recorded lighting conditions to satisfy the value threshold. For this specific task, the important information is therefore intensity contrast, not colour information.

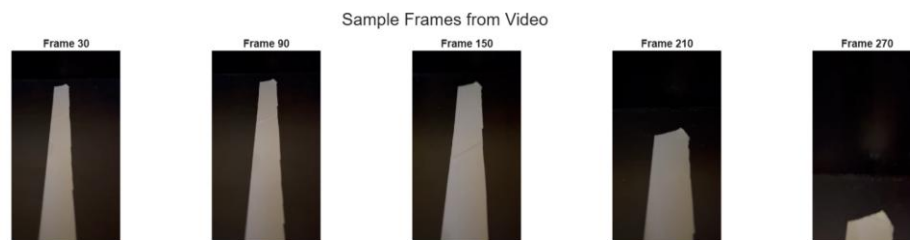


Figure 1: Sample frames extracted from the recorded video.

2. How the final code works

The final code follows the sequence below:

- **Load the video:** The script opens Video.mp4 using VideoReader, reads the frame rate, duration, number of frames and frame size.
- **Extract sample frames:** A few frames are selected from the video so that the segmentation quality can be visually checked.
- **Resize the frame:** Each frame is downscaled to 25% of the original resolution. This reduces the number of pixels processed and improves speed.
- **Convert to grayscale:** The final pipeline converts the resized RGB frame to a single grayscale image.
- **Apply a brightness threshold:** Pixels brighter than the threshold are classified as tape. In the tested code the threshold is 100 on the 0-255 grayscale scale.
- **Remove small noise blobs:** bwareaopen removes small connected components with area below the minimum area threshold.
- **Close small gaps:** Morphological closing with a disk-shaped structuring element fills small breaks and improves the continuity of the detected tape region.

- **Use the largest detected blob:** regionprops is used to find connected regions. The largest area is selected as the tape, which makes the method more robust against small bright reflections or noise.
- **Calculate the centroid:** The centroid of the selected tape region gives the detected line position in the image.
- **Calculate the error signal:** The centroid x-position is compared with the image centre to produce a normalised steering-style error signal.

The final detection is shown by overlaying the detected mask on the original sample frames. The green overlay shows that the grayscale method consistently captures the visible white tape, including when the tape appears lower in the frame near the end of the video.

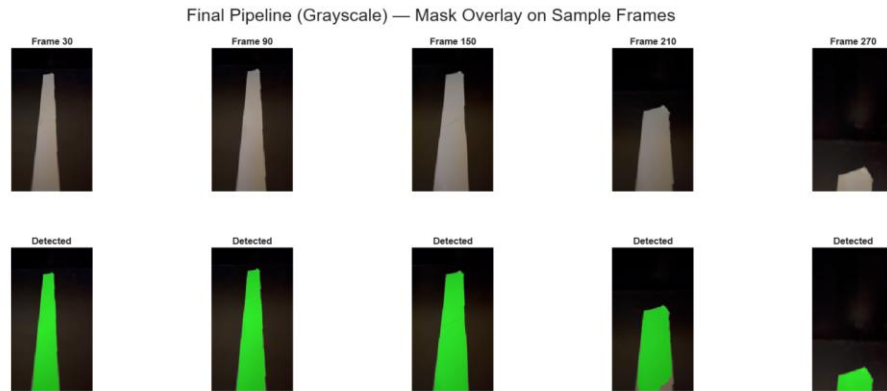


Figure 2: Final grayscale pipeline with the detected tape mask shown in green.

3. RGB versus HSV line segmentation

The code compares grayscale thresholding, RGB-brightness thresholding and HSV thresholding. The grayscale method thresholds the intensity after converting the frame to grayscale. The RGB method calculates an average brightness from the red, green and blue channels and thresholds that value. The HSV method converts the image to hue, saturation and value, then attempts to detect white tape using low saturation and high brightness.

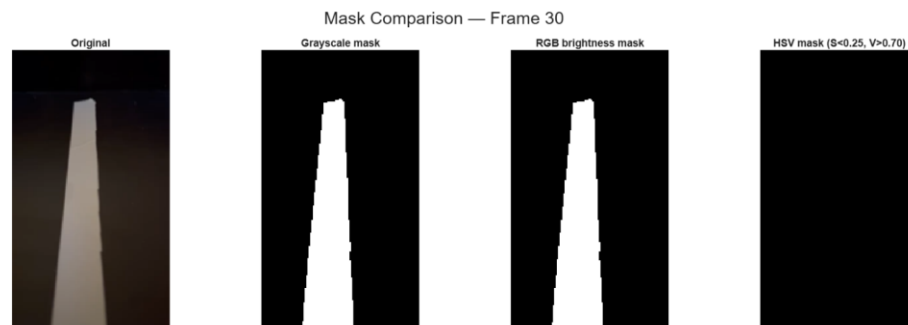


Figure 3: Visual comparison of grayscale, RGB brightness and HSV masks for Frame 30.

The visual comparison shows that grayscale and RGB-brightness segmentation produce almost the same mask. This is expected because both approaches are essentially measuring the same property: the tape is much brighter than the black floor. The HSV result is black for the tested frame, meaning that no significant tape region was detected with the selected HSV thresholds.

The quantitative comparison supports this observation. The approximate values read from the generated plots are summarised below. The exact numbers may vary slightly depending on the video frame, the resize factor and the computer used to run the timing test.

Metric	Grayscale	RGB brightness	HSV
Mean SNR over sample frames	About 8.8-9.0	About 8.5-8.8	Approximately 0
Mean mask coverage	About 18-19%	About 18%	Approximately 0%
50-frame timing at 25% resolution	1.53 ms/frame	1.30 ms/frame	1.24 ms/frame
Estimated processing speed	~655 FPS	~772 FPS	~806 FPS
Detection quality	Good	Good	Poor with chosen thresholds

The HSV method was not selected as the final method because the thresholds were too strict for this recording. White objects are normally expected to have low saturation and high value in HSV, but the recorded tape appears dimmer because of uneven lighting, distance from the camera and the dark scene. As a result, many tape pixels do not pass the $V > 0.70$ condition. The HSV approach could be improved by lowering the value threshold or by using adaptive thresholds, but this would add extra tuning without providing a clear advantage for this video.

For this particular task, there is not a meaningful benefit in using a full colour-space method. The target line is white and the background is dark, so the separation is dominated by brightness contrast. That is why grayscale and RGB-brightness segmentation give very similar masks, while grayscale is simpler and more suitable for a final real-time pipeline.

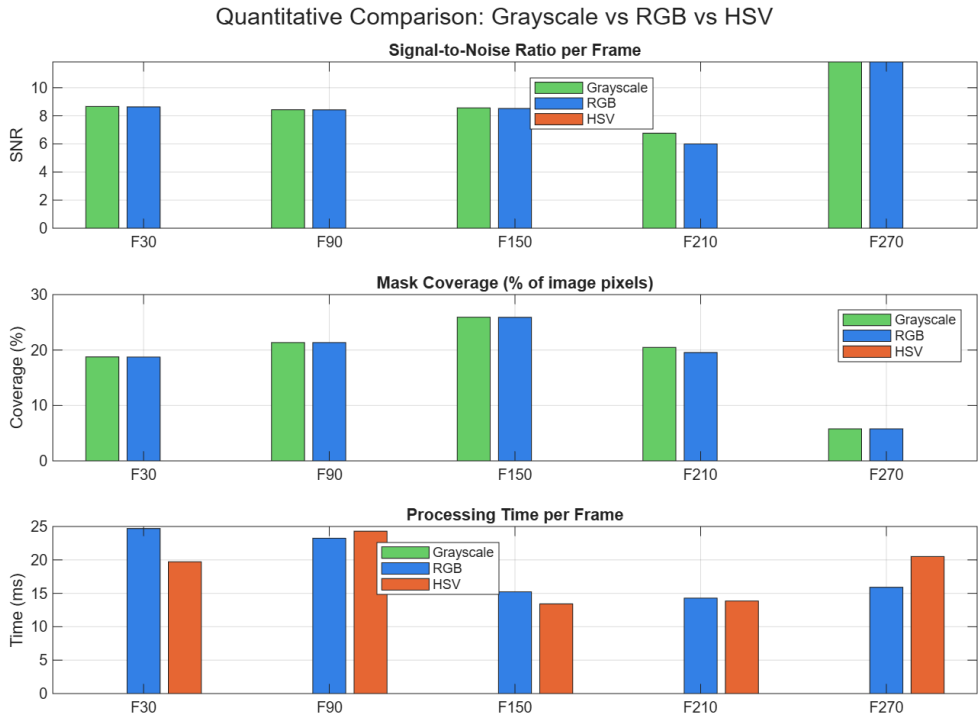


Figure 4: Quantitative comparison of signal-to-noise ratio, mask coverage and processing time.

4. Speed improvements included in the final code

Several choices were included to make the code faster and more suitable for real-time implementation:

- **Downscaling to 25% resolution:** The variable `SCALE = 0.25` reduces the number of pixels to about one-sixteenth of the original full-resolution image. This gives a large speed improvement while preserving enough line shape information.
- **Using grayscale for the final pipeline:** The final method avoids unnecessary colour-space conversion. A single grayscale channel is enough because the tape is identified mainly by brightness.
- **Processing only the region of interest:** The error calculation uses only the lower part of the image by setting `ROI_TOP = 0.5`. This ignores the top half of the frame where the line is less useful for steering and reduces the number of pixels processed.
- **Simple thresholding:** A fixed grayscale threshold is computationally cheaper than more complex segmentation methods.
- **Small-object removal and morphological closing:** These operations clean the mask without requiring heavier machine-learning or edge-detection methods.
- **Largest-component selection:** Selecting the largest blob reduces false detections from small bright spots while still being computationally simple.

The speed comparison over 50 frames shows that all three approaches are fast at 25% resolution. The measured values were approximately 1.53 ms/frame for grayscale, 1.30 ms/frame for RGB brightness and 1.24 ms/frame for HSV. These values correspond to several hundred frames per second on the test computer. In practice, the full system speed would also depend on camera capture time, hardware, storage, display and any robot-control code running at the same time.

Even though the measured HSV timing appears slightly faster in the 50-frame test, the HSV method did not produce a useful mask with the chosen thresholds. Therefore speed alone should not be used to choose the final method. The grayscale method was selected because it gives reliable detection with a simple and explainable implementation.

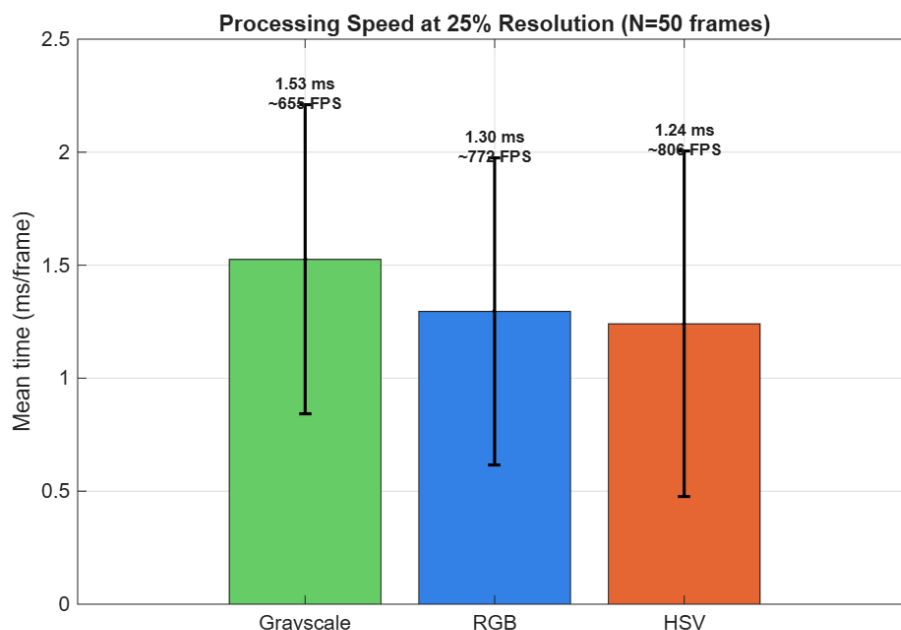


Figure 5: Processing speed comparison over 50 frames at 25% resolution.

5. Error metric and line position over time

After the tape is segmented, the code calculates the centroid of the largest detected tape region. Only the horizontal centroid position is used for the error metric, because the aim is to determine whether the line is left or right of the image centre.

The normalised error is calculated as:

$$\text{error} = (\text{cx} - \text{image_width}/2) / (\text{image_width}/2)$$

In this equation, cx is the x-coordinate of the detected tape centroid. A value of 0 means that the tape is in the centre of the image. A negative value means the line is to the left of the centre, and a positive value means the line is to the right. The scale is normalised so that -1 represents the far left side of the image and +1 represents the far right side.

Error value	Meaning
-1	Line is near the far left of the image
0	Line is centred
+1	Line is near the far right of the image

The plotted error signal shows that the line remains close to the image centre for most of the video. The mean error is -0.019, which indicates a slight average offset to the left. The standard deviation is 0.091, showing that the detected line position does not vary strongly during the recorded sequence. The smoothed signal follows the same trend as the raw signal but removes small frame-to-frame fluctuations, making it more suitable for later use in a control system.

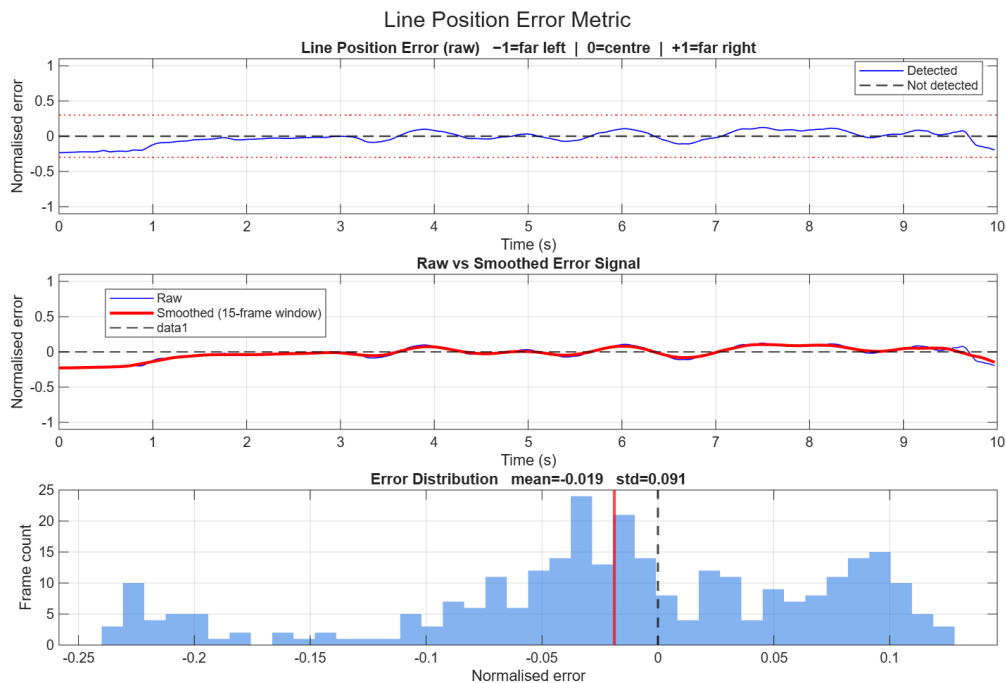


Figure 6: Raw and smoothed line-position error over time, with the error distribution.

The centroid path plot gives another view of the line movement. The dashed vertical line indicates the image centre. The coloured points show the detected centroid location over time. Most detections remain close to the centre line, confirming the result from the error plot.

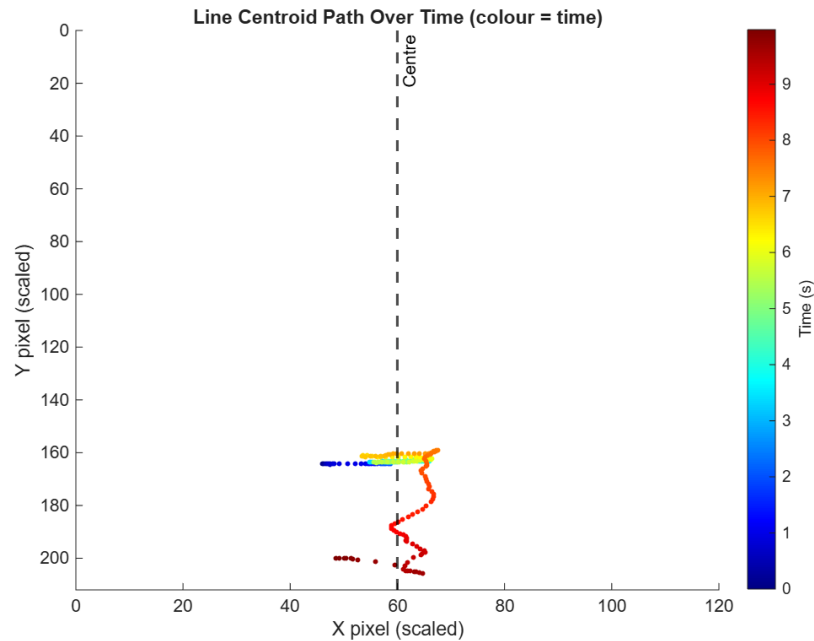


Figure 7: Detected line centroid path over time.

6. Limitations and recommended final adjustments

The current pipeline works well for this controlled video because the white tape has strong contrast against the dark floor. However, fixed thresholds can become less reliable if the lighting changes strongly, if the floor becomes reflective, or if other bright objects appear in the frame.

For a more robust final implementation, the code could include adaptive thresholding, automatic threshold calibration from the first few frames, or a dynamic region of interest. It would also be useful to test the method with different lighting conditions, different tape widths, different camera heights and robot motion.

One coding detail to check in the final MATLAB script is that timing values for grayscale, RGB and HSV should be measured in separate tic/toc blocks. This ensures that the plotted processing-time comparison is fair and that no method is accidentally assigned a zero timing value.

7. Conclusion

The best final version of the code uses a grayscale-based segmentation pipeline. This method was selected because it detects the white tape reliably, is simple to explain, and avoids unnecessary colour processing. RGB-brightness segmentation gives almost the same result because it also measures intensity contrast. HSV segmentation did not work well with the selected thresholds for this video, mainly because the tape was not bright enough in the recorded lighting to pass the high-value threshold.

The final pipeline also calculates a useful normalised error metric. For the recorded video, the mean error was -0.019 with a standard deviation of 0.091, meaning that the line stayed close to the centre with only a small left offset. This error signal is suitable as an input for a future robot steering or line-following controller.